Project Checkpoint 3

Goal:

- Use more data to train the network
- Tune architecture, learning rate, epoch count, batch size, and L2 regularization
- Re-initialize the model for every trial
- Compare validation performance fairly
- Defend the final chosen model

In [1]:
```
import warnings
warnings.filterwarnings("ignore", category=UserWarning)

import copy
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

import torch
import torch.nn as nn
import torchani
```

Device

In [5]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("Device:", device)
```

```
Device: cuda
```

Dataset Loading

In [6]:
```
def load_ani_dataset(dspath):
    species_order = ['H', 'C', 'N', 'O']
    energy_shifter = torchani.utils.EnergyShifter(None)

    dataset = torchani.data.load(dspath)
    dataset = dataset.subtract_self_energies(energy_shifter, species_order)
    dataset = dataset.species_to_indices(species_order)
    dataset = dataset.shuffle()
    return dataset
```

AEV Computer

In [8]:
```
def init_aev_computer():
    Rcr = 5.2
```

```
    Rca = 3.5

    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)

    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
        0.19634954, 0.58904862, 0.98174770, 1.37444680,
        1.76714590, 2.15984490, 2.55254400, 2.94524300
    ], dtype=torch.float, device=device)

    return torchani.AEVComputer(
        Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, 4
    )

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print("AEV dimension:", aev_dim)
```

```
AEV dimension: 384
```

Flexible atomic network

In [10]:

```
class AtomicNet(nn.Module):
    def __init__(self, input_dim, hidden_layers, activation="relu"):
        super().__init__()

        layers = []
        prev_dim = input_dim

        if activation == "relu":
            act_layer = nn.ReLU
        elif activation == "tanh":
            act_layer = nn.Tanh
        else:
            raise ValueError("Unsupported activation. Use 'relu' or 'tanh'.")

        for hidden_dim in hidden_layers:
            layers.append(nn.Linear(prev_dim, hidden_dim))
            layers.append(act_layer())
            prev_dim = hidden_dim

        layers.append(nn.Linear(prev_dim, 1))
        self.layers = nn.Sequential(*layers)

    def forward(self, x):
        return self.layers(x)
```

Hyperparameters

In [11]:
```
def build_model(hidden_layers, activation="relu"):
    net_H = AtomicNet(aev_dim, hidden_layers, activation=activation)
    net_C = AtomicNet(aev_dim, hidden_layers, activation=activation)
    net_N = AtomicNet(aev_dim, hidden_layers, activation=activation)
    net_O = AtomicNet(aev_dim, hidden_layers, activation=activation)

    ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])

    model = nn.Sequential(
        aev_computer,
        ani_net
    ).to(device)

    return model
```

Trainer

In [12]:
```
class ANITrainer:
    def __init__(self, model, batch_size, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.batch_size = batch_size
        self.optimizer = torch.optim.Adam(
            self.model.parameters(),
            lr=learning_rate,
            weight_decay=l2
        )
        self.epoch = epoch

    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        print("Initialize training data...")
        train_data_loader = train_data.collate(self.batch_size).cache()

        loss_func = nn.MSELoss()

        train_loss_list = []
        val_loss_list = []
        lowest_val_loss = np.inf
        best_weights = None

        for i in tqdm(range(self.epoch), leave=True):
            self.model.train()
            train_epoch_loss = 0.0
            train_epoch_count = 0

            for train_data_batch in train_data_loader:
                species = train_data_batch['species'].to(device)
```

```
                coords = train_data_batch['coordinates'].to(device)
                true_energies = train_data_batch['energies'].to(device).float()

                _, pred_energies = self.model((species, coords))
                batch_loss = loss_func(pred_energies, true_energies)

                self.optimizer.zero_grad()
                batch_loss.backward()
                self.optimizer.step()

                batch_importance = species.shape[0]
                train_epoch_loss += batch_loss.item() * batch_importance
                train_epoch_count += batch_importance

            train_epoch_loss /= train_epoch_count
            val_epoch_loss = self.evaluate(val_data, draw_plot=False)

            train_loss_list.append(train_epoch_loss)
            val_loss_list.append(val_epoch_loss)

            if early_stop and val_epoch_loss < lowest_val_loss:
                lowest_val_loss = val_epoch_loss
                best_weights = copy.deepcopy(self.model.state_dict())

        if draw_curve:
            fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
            ax.set_yscale("log")
            ax.plot(train_loss_list, label="Train")
            ax.plot(val_loss_list, label="Validation")
            ax.set_xlabel("Epoch")
            ax.set_ylabel("MSE Loss")
            ax.legend()

        if early_stop and best_weights is not None:
            self.model.load_state_dict(best_weights)

        return train_loss_list, val_loss_list

    def evaluate(self, data, draw_plot=False):
        data_loader = data.collate(self.batch_size).cache()

        loss_func = nn.MSELoss()
        total_loss = 0.0
        total_count = 0

        if draw_plot:
            true_energies_all = []
            pred_energies_all = []

        self.model.eval()

        with torch.no_grad():
            for batch_data in data_loader:
                species = batch_data['species'].to(device)
                coords = batch_data['coordinates'].to(device)
                true_energies = batch_data['energies'].to(device).float()
```

```
            _, pred_energies = self.model((species, coords))
            batch_loss = loss_func(pred_energies, true_energies)

            batch_importance = species.shape[0]
            total_loss += batch_loss.item() * batch_importance
            total_count += batch_importance

            if draw_plot:
                true_energies_all.append(true_energies.detach().cpu().numpy().f
                pred_energies_all.append(pred_energies.detach().cpu().numpy().f

    avg_loss = total_loss / total_count

    if draw_plot:
        true_energies_all = np.concatenate(true_energies_all)
        pred_energies_all = np.concatenate(pred_energies_all)

        hartree2kcalmol = 627.5094738898777
        mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2

        fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
        ax.scatter(true_energies_all, pred_energies_all,
                   label=f"MAE: {mae:.2f} kcal/mol", s=2)
        ax.set_xlabel("Ground Truth Energy (Hartree)")
        ax.set_ylabel("Predicted Energy (Hartree)")

        xmin, xmax = ax.get_xlim()
        ymin, ymax = ax.get_ylim()
        vmin, vmax = min(xmin, ymin), max(xmax, ymax)
        ax.set_xlim(vmin, vmax)
        ax.set_ylim(vmin, vmax)
        ax.plot([vmin, vmax], [vmin, vmax], color='red')
        ax.legend()

        print(f"MAE: {mae:.4f} kcal/mol")

    return avg_loss

def evaluate_mae_kcalmol(self, data):
    data_loader = data.collate(self.batch_size).cache()

    true_energies_all = []
    pred_energies_all = []

    self.model.eval()

    with torch.no_grad():
        for batch_data in data_loader:
            species = batch_data['species'].to(device)
            coords = batch_data['coordinates'].to(device)
            true_energies = batch_data['energies'].to(device).float()

            _, pred_energies = self.model((species, coords))

            true_energies_all.append(true_energies.detach().cpu().numpy().flatt
```

```
                pred_energies_all.append(pred_energies.detach().cpu().numpy().flatt

        true_energies_all = np.concatenate(true_energies_all)
        pred_energies_all = np.concatenate(pred_energies_all)

        hartree2kcalmol = 627.5094738898777
        mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcal
        return mae
```

Load Data

In [13]:
```
dataset_path = "./ani_gdb_s01_to_s04.h5"
dataset = load_ani_dataset(dataset_path)

# Keep the same split for fair comparisons inside this notebook
train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)
print("Dataset loaded and split.")
```

```
Dataset loaded and split.
```

Experiment Runner

In [14]:
```
def run_experiment(
    train_data,
    val_data,
    hidden_layers,
    batch_size,
    learning_rate,
    epoch,
    l2,
    activation="relu",
    draw_curve=False
):
    # IMPORTANT: re-initialize model every trial
    model = build_model(hidden_layers=hidden_layers, activation=activation)

    trainer = ANITrainer(
        model=model,
        batch_size=batch_size,
        learning_rate=learning_rate,
        epoch=epoch,
        l2=l2
    )

    train_losses, val_losses = trainer.train(
        train_data,
        val_data,
        early_stop=True,
        draw_curve=draw_curve
    )

    final_val_mse = trainer.evaluate(val_data, draw_plot=False)
    final_val_mae = trainer.evaluate_mae_kcalmol(val_data)
```

```
    result = {
        "hidden_layers": hidden_layers,
        "activation": activation,
        "batch_size": batch_size,
        "learning_rate": learning_rate,
        "epoch": epoch,
        "l2": l2,
        "final_val_mse": final_val_mse,
        "final_val_mae_kcalmol": final_val_mae,
        "train_losses": train_losses,
        "val_losses": val_losses,
        "trainer": trainer
    }
    return result
```

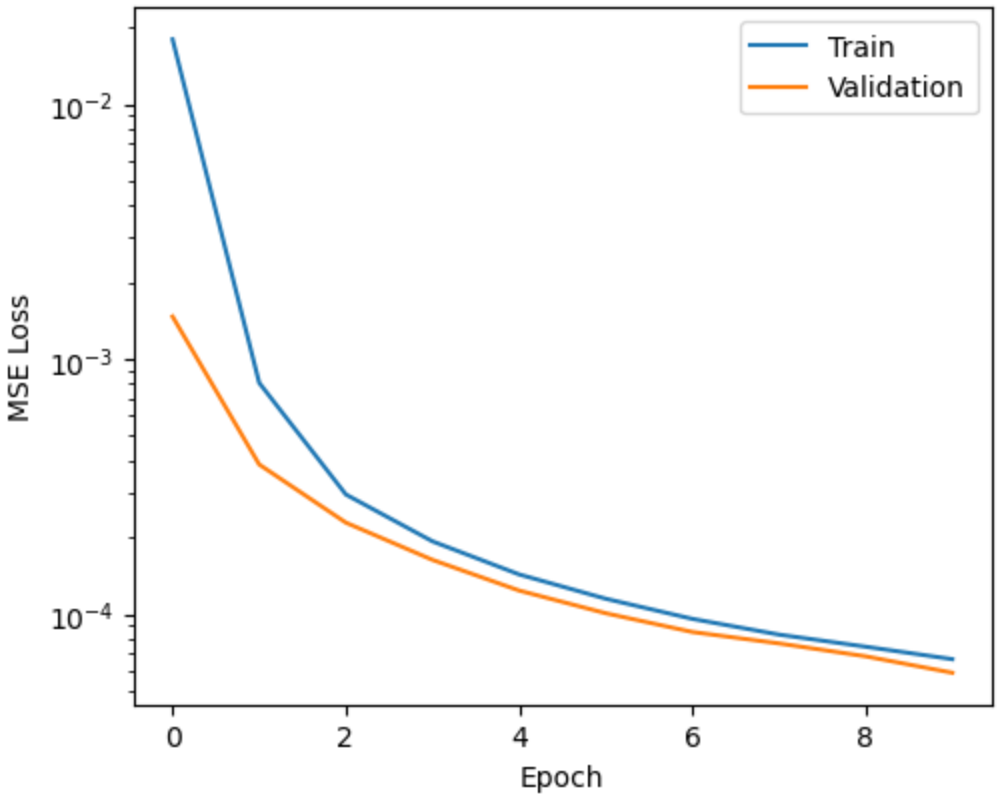
Experiments

In [15]:
```
results = []

# Baseline trial
results.append(run_experiment(
    train_data=train_data,
    val_data=val_data,
    hidden_layers=[128],
    batch_size=8192,
    learning_rate=1e-3,
    epoch=10,
    l2=1e-5,
    activation="relu",
    draw_curve=True
))

# Architecture trial: deeper network
results.append(run_experiment(
    train_data=train_data,
    val_data=val_data,
    hidden_layers=[128, 64],
    batch_size=8192,
    learning_rate=1e-3,
    epoch=10,
    l2=1e-5,
    activation="relu",
    draw_curve=False
))

# Architecture trial: wider network
results.append(run_experiment(
    train_data=train_data,
    val_data=val_data,
    hidden_layers=[256, 128],
    batch_size=8192,
    learning_rate=1e-3,
    epoch=10,
```

```
    l2=1e-5,
    activation="relu",
    draw_curve=False
))

# Learning rate trial
results.append(run_experiment(
    train_data=train_data,
    val_data=val_data,
    hidden_layers=[128, 64],
    batch_size=8192,
    learning_rate=1e-4,
    epoch=15,
    l2=1e-5,
    activation="relu",
    draw_curve=False
))

# Regularization trial
results.append(run_experiment(
    train_data=train_data,
    val_data=val_data,
    hidden_layers=[128, 64],
    batch_size=8192,
    learning_rate=1e-3,
    epoch=10,
    l2=1e-4,
    activation="relu",
    draw_curve=False
))

# Batch-size trial
results.append(run_experiment(
    train_data=train_data,
    val_data=val_data,
    hidden_layers=[128, 64],
    batch_size=4096,
    learning_rate=1e-3,
    epoch=10,
    l2=1e-5,
    activation="relu",
    draw_curve=False
))
```

```
Sequential - Number of parameters: 197636
Initialize training data...
100%|██████████| 10/10 [01:17<00:00,  7.73s/it]
Sequential - Number of parameters: 230404
Initialize training data...
100%|██████████| 10/10 [01:00<00:00,  6.00s/it]
Sequential - Number of parameters: 526340
Initialize training data...
100%|██████████| 10/10 [01:03<00:00,  6.35s/it]
Sequential - Number of parameters: 230404
Initialize training data...
```

```
100%|██████████| 15/15 [01:29<00:00,  5.95s/it]
Sequential - Number of parameters: 230404
Initialize training data...
100%|██████████| 10/10 [00:59<00:00,  5.93s/it]
Sequential - Number of parameters: 230404
Initialize training data...
100%|██████████| 10/10 [01:00<00:00,  6.05s/it]
```



Summary

In [16]:
```
print("\nCheckpoint 3 experiment summary:")
for idx, res in enumerate(results):
    print(
        f"Trial {idx + 1}: "
        f"layers={res['hidden_layers']}, "
        f"activation={res['activation']}, "
        f"batch={res['batch_size']}, "
        f"lr={res['learning_rate']}, "
        f"epoch={res['epoch']}, "
        f"l2={res['l2']} | "
        f"Val MSE={res['final_val_mse']:.6f}, "
        f"Val MAE={res['final_val_mae_kcalmol']:.4f} kcal/mol"
    )
```

```
Checkpoint 3 experiment summary:
Trial 1: layers=[128], activation=relu, batch=8192, lr=0.001, epoch=10, l2=1e-05 | V
al MSE=0.000059, Val MAE=2.8649 kcal/mol
Trial 2: layers=[128, 64], activation=relu, batch=8192, lr=0.001, epoch=10, l2=1e-05
| Val MSE=0.000027, Val MAE=2.0464 kcal/mol
Trial 3: layers=[256, 128], activation=relu, batch=8192, lr=0.001, epoch=10, l2=1e-0
5 | Val MSE=0.000019, Val MAE=1.7352 kcal/mol
Trial 4: layers=[128, 64], activation=relu, batch=8192, lr=0.0001, epoch=15, l2=1e-0
5 | Val MSE=0.000040, Val MAE=2.2696 kcal/mol
Trial 5: layers=[128, 64], activation=relu, batch=8192, lr=0.001, epoch=10, l2=0.000
1 | Val MSE=0.000061, Val MAE=2.6186 kcal/mol
Trial 6: layers=[128, 64], activation=relu, batch=4096, lr=0.001, epoch=10, l2=1e-05
| Val MSE=0.000015, Val MAE=1.5704 kcal/mol
```

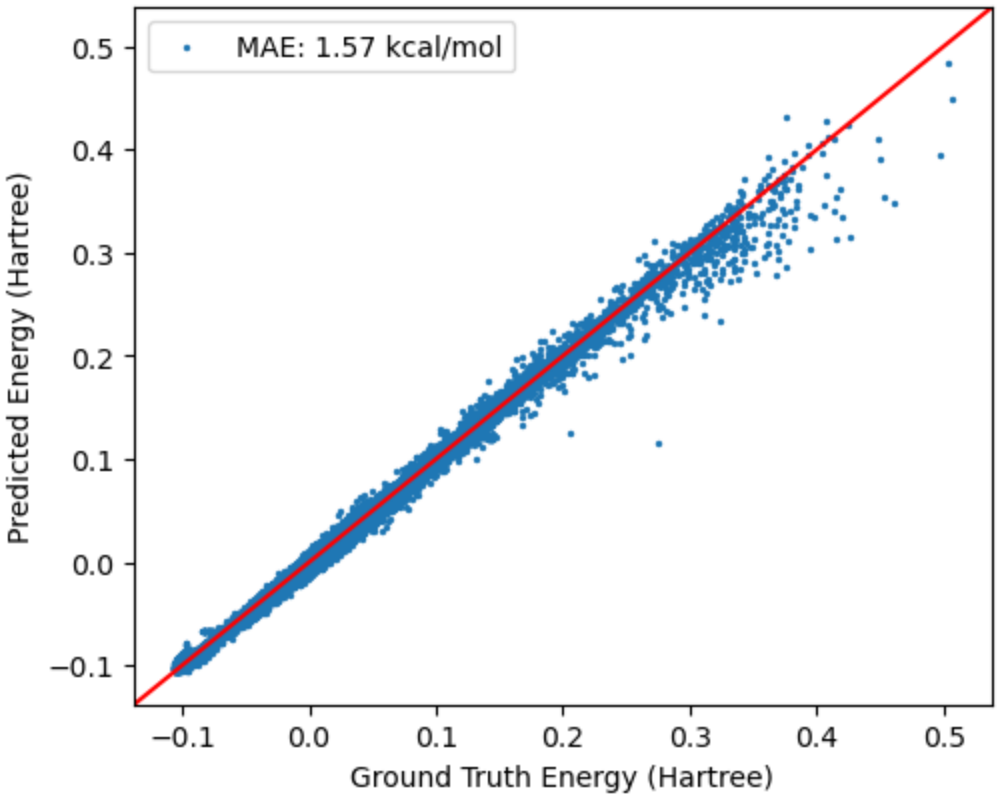
Model Selection

In [17]:
```python
best_result = min(results, key=lambda x: x["final_val_mae_kcalmol"])

print("\nBest validation configuration:")
print(f"hidden_layers = {best_result['hidden_layers']}")
print(f"activation    = {best_result['activation']}")
print(f"batch_size    = {best_result['batch_size']}")
print(f"learning_rate = {best_result['learning_rate']}")
print(f"epoch         = {best_result['epoch']}")
print(f"l2            = {best_result['l2']}")
print(f"Val MAE       = {best_result['final_val_mae_kcalmol']:.4f} kcal/mol")
```

```
Best validation configuration:
hidden_layers = [128, 64]
activation    = relu
batch_size    = 4096
learning_rate = 0.001
epoch         = 10
l2            = 1e-05
Val MAE       = 1.5704 kcal/mol
```

In [18]:
```python
best_trainer = best_result["trainer"]

print("\nBest model test-set evaluation:")
test_mse = best_trainer.evaluate(test_data, draw_plot=True)
test_mae = best_trainer.evaluate_mae_kcalmol(test_data)

print(f"Test MSE: {test_mse:.6f}")
print(f"Test MAE: {test_mae:.4f} kcal/mol")
```

```
Best model test-set evaluation:
MAE: 1.5749 kcal/mol
Test MSE: 0.000016
Test MAE: 1.5749 kcal/mol
```



The results show that the trials are actually differentiating model choices in a meaningful way, not producing random noise. In particular, the best validation configuration is [128, 64] with batch size 4096, learning rate 1e-3, epoch 10, and l2=1e-5, with validation MAE 1.5704 kcal/mol, and the test MAE is 1.5749 kcal/mol, which is very close to validation and therefore looks stable rather than suspiciously overfit.

In [ ]: